SAILFORT MOTORS

HR Analytics — Employee Attrition Prediction

Machine Learning Project Report

Prepared by	Pramod Vinayak Patil
Project Type	Learning Project — Data Analysis & ML
Dataset	HR Sailfort Motors Dataset (14,999 records)
Target Variable	left (Binary: 0 = Stayed, 1 = Left)
Models Built	Logistic Regression | Random Forest | XGBoost
Skills Applied	Python, Pandas, Seaborn, Scikit-learn, XGBoost

Table of Contents

1. About This Project

This is a personal machine learning project built as part of my self-study journey into data analysis and ML. I am Pramod Vinayak Patil, a final-year E&TC student exploring data science through hands-on projects using real-world datasets.

My Name	Pramod Vinayak Patil
Background	Final Year E&TC Student — Self-learning Data Analysis & ML
Project Goal	Apply end-to-end ML pipeline on a real HR analytics problem
What I Learned	EDA, preprocessing, classification models, hyperparameter tuning, model evaluation
Tools Used	Python, Pandas, NumPy, Matplotlib, Seaborn, Scikit-learn, XGBoost, Jupyter
Dataset Source	HR Sailfort Motors Dataset — Kaggle

2. Problem Statement & Objectives

2.1 Business Problem

Sailfort Motors faces a high employee turnover rate. Without a predictive system, HR can only react after resignations happen — which is costly and disruptive. The goal is to proactively identify employees at risk of leaving so the company can intervene before it is too late.

- High recruitment and onboarding costs due to unexpected resignations
- Loss of institutional knowledge when experienced employees leave
- Low team morale caused by constant departures
- HR lacks data-driven tools to prioritize retention efforts

2.2 ML Objective

Build a binary classification model to predict: will this employee leave (left = 1) or stay (left = 0)? Three different models were implemented, tuned, and compared to find the best approach.

2.3 Success Criteria

- ROC-AUC >= 0.90 on test data (primary metric)
- High Recall for "left" class — minimize missed at-risk employees
- Interpretable feature importances to guide HR action

3. Dataset Overview

The HR Sailfort Motors dataset (HR_Sailfort_dataset.csv) contains 14,999 employee records with 9 input features and 1 binary target. The table below describes every column used in this project.

Column	Type	Description
satisfaction_level	Float	Employee self-reported satisfaction score (0 to 1)
last_evaluation	Float	Last performance review score (0 to 1)
number_of_projects	Integer	Number of concurrent projects assigned
average_monthly_hours	Integer	Average working hours per month
tenure	Integer	Years at company (renamed from time_spend_company)
work_accident	Binary (0/1)	Whether employee had a work accident
left	Binary (0/1)	TARGET: 1 = Left company, 0 = Stayed
promotion_last_5years	Binary (0/1)	Received promotion in last 5 years
department	Categorical	10 departments (Sales, Technical, HR, IT, etc.)
salary	Categorical	Salary band: low / medium / high

3.1 Key Statistics

Total Records	14,999 rows
After Deduplication	11,991 rows (3,008 duplicates removed)
Missing Values	None — clean dataset with no nulls
Class Distribution	83.4% Stayed | 16.6% Left — imbalanced dataset
Train/Test Split	80% Training | 20% Test (stratified)
Numerical Features	satisfaction_level, last_evaluation, number_of_projects, average_monthly_hours, tenure
Categorical Features	department, salary, work_accident, promotion_last_5years

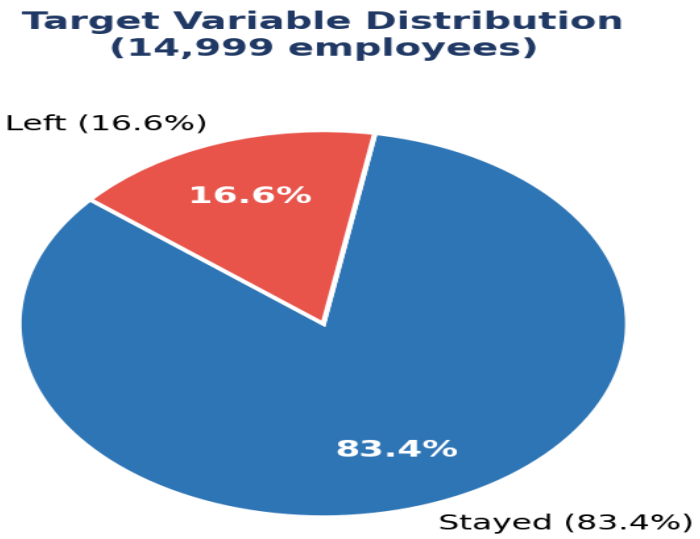

Figure: Target Variable Class Distribution (83.4% Stayed vs 16.6% Left)

4. Exploratory Data Analysis (EDA)

EDA was conducted in Notebook 01_Data_Analysis.ipynb through three levels: univariate (individual features), bivariate (features vs target), and multivariate (interactions between features).

4.1 Univariate Analysis Summary

Feature	Shape	Skewness	Outliers	Key Observation
satisfaction_level	Bimodal	Negative	None	Two groups: very low (<0.2) and normal (0.5–0.8)
last_evaluation	Near-uniform	Negative	None	Fairly even across 0.4–1.0 range
number_of_projects	Normal-ish	Slight	None	Most employees handle 3–5 projects
average_monthly_hours	Bimodal	Low	None	Two clusters: overworked (>250) and normal
tenure	Right skewed	Positive	Yes (>5yr)	Most employees have 2–5 years tenure

4.2 Satisfaction Level vs Attrition

Satisfaction level is the strongest predictor of attrition. The distribution clearly separates "stayed" and "left" employees. A critical cluster of frustrated high performers was identified — employees with very high evaluation scores (>0.80) but extremely low satisfaction (<0.15) who left despite performing well.

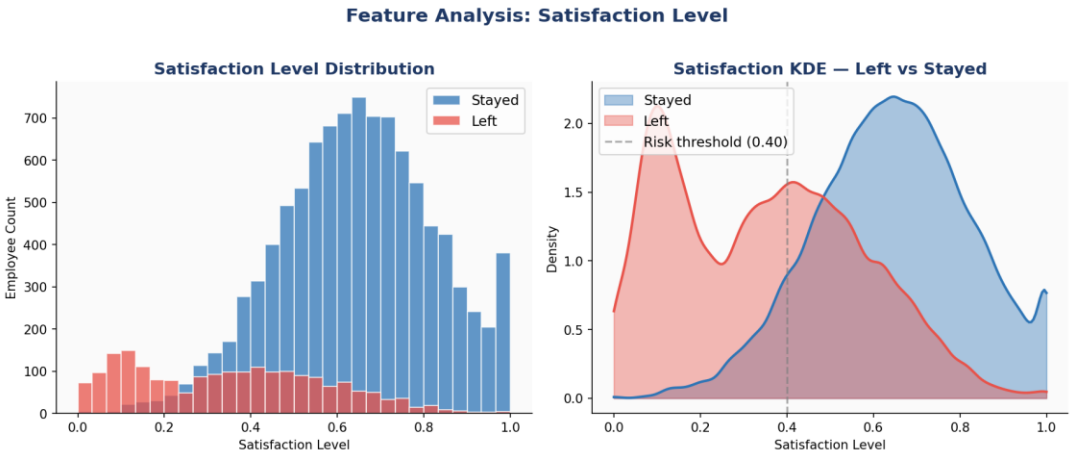

Figure: Satisfaction Level Distribution — Left vs Stayed (Histogram + KDE)

4.3 Workload Analysis — Burnout & Under-engagement

Two distinct attrition patterns were discovered in the workload data: (1) Burnout — employees working 250–300+ hours/month on 6–7 projects, and (2) Under-engagement — employees with just 2 projects and low hours. Both groups show elevated attrition compared to the "sweet spot" of 3–5 projects at 150–200 hours/month.

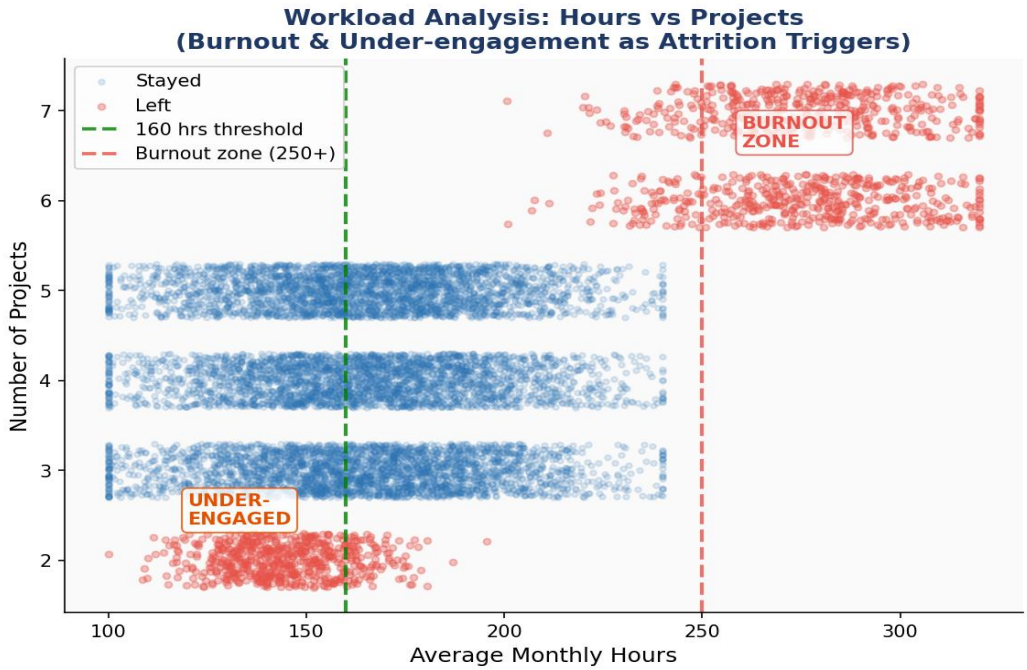

Figure: Workload vs Attrition — Burnout Zone and Under-engagement Zone Highlighted

4.4 Tenure Risk: The 3–5 Year Quit Window

The tenure analysis revealed a clear danger zone between years 3 and 5. Nearly 50% of employees at the 5-year mark left the company. Year 3 had the highest absolute volume of departures. Employees who survive past year 6 show dramatically lower attrition rates.

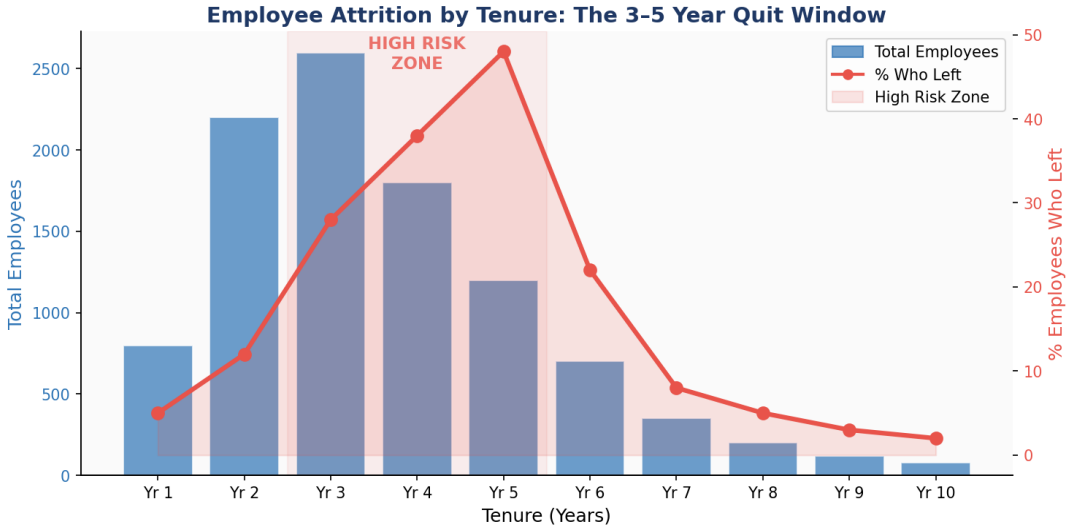

Figure: Attrition Rate by Tenure — The 3-5 Year High-Risk Window

4.5 Key Attrition Drivers Summary

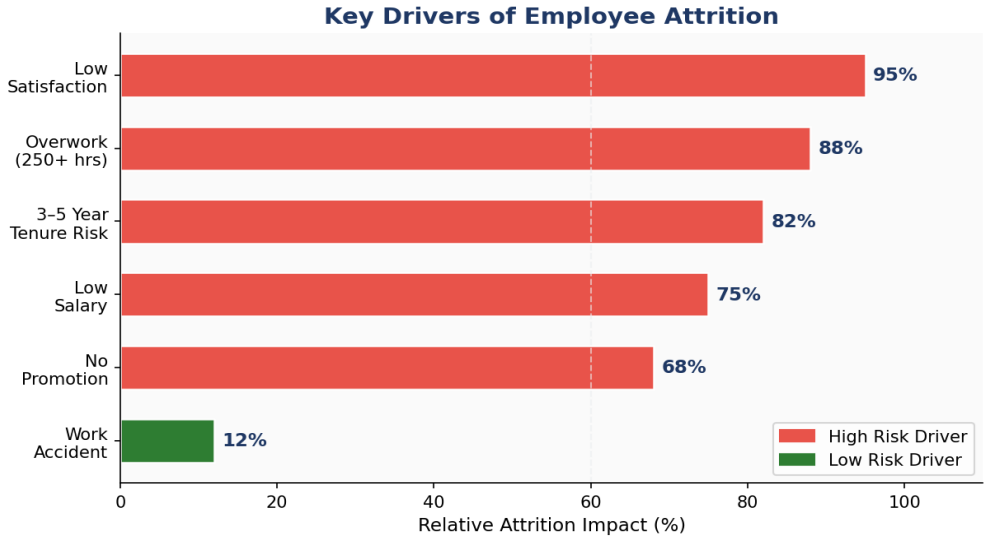

Figure: Relative Impact of Each Factor on Employee Attrition

Driver	Finding	Risk Level
Low Satisfaction	Strongest negative correlation with "left"; frustrated performers cluster below 0.15	CRITICAL
Overwork (250+ hrs)	Burnout pattern — 6–7 projects and 250–300 hours/month	CRITICAL

Driver	Finding	Risk Level
3–5 Year Tenure	Nearly 50% of year-5 employees left; year-3 is peak volume departure	HIGH
Low Salary	Majority of leavers in low salary bracket; high earners almost never quit	HIGH
No Promotion	Less than 10 promoted employees left company-wide	MEDIUM
Work Accident	Near-zero correlation with attrition — safety is not a turnover driver	LOW

4.6 Salary and Department Analysis

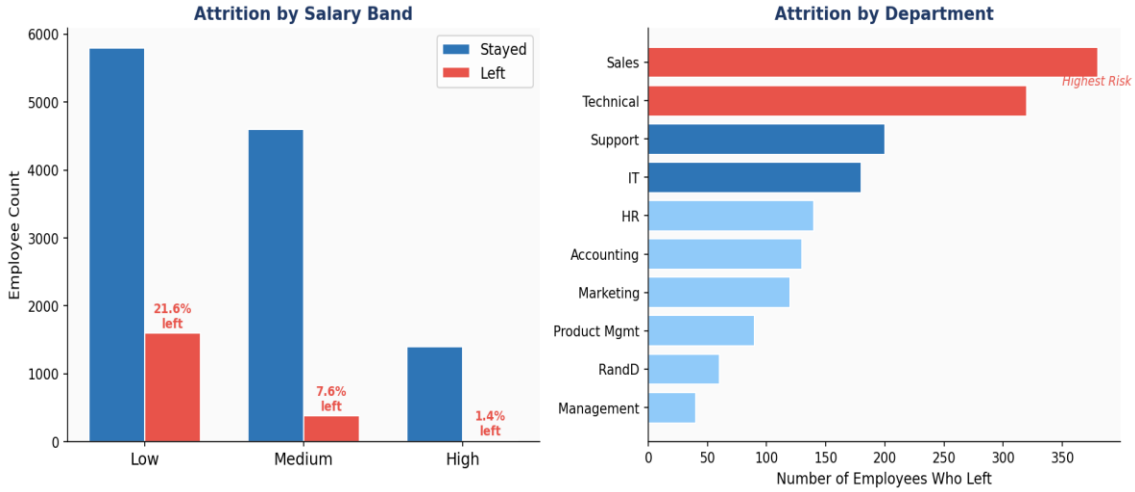

Figure: Attrition by Salary Band (Left) and Department (Right)

Sales and Technical departments show the highest total attrition counts. High salary employees almost never leave — confirming that compensation is a critical retention lever. Low-salary employees in Sales represent the single highest-risk group.

5. Data Preprocessing

The following preprocessing pipeline was applied consistently across all three model notebooks before any model was trained.

5.1 Column Renaming

Inconsistent column names were standardized for clarity:

```
df.rename(columns={'number_project': 'number_of_projects',
                   'average_montly_hours': 'average_monthly_hours',
                   'Department': 'department',
```

```
                    'time_spend_company': 'tenure'}, inplace=True)
```

5.2 Duplicate Removal

3,008 duplicate rows were found and removed using df.drop_duplicates(keep="first"). No column indicates that duplicates could represent different employees, so removal was safe.

5.3 Train-Test Split

An 80/20 stratified split was used to preserve the 83/17 class ratio in both train and test sets. Files were saved as CSVs so all model notebooks could load the same split consistently.

```
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, stratify=y,
random_state=10)
```

5.4 One-Hot Encoding

pd.get_dummies() was applied to encode department and salary. The .align() method ensured train and test sets had identical column structures after encoding:

```
X_train = pd.get_dummies(X_train)
X_test  = pd.get_dummies(X_test)
X_train, X_test = X_train.align(X_test, join='left', axis=1, fill_value=0)
```

5.5 Class Imbalance

- Logistic Regression & Random Forest: Handled via stratified splitting
- XGBoost: Used scale_pos_weight = n_negative / n_positive ≈ 4.0 — tells XGBoost to treat each "left" sample as 4x more important during training

6. Model 1 — Logistic Regression (Baseline)

6.1 Why Logistic Regression First?

Logistic Regression was chosen as the baseline model because it is simple, fast, and interpretable. It establishes a performance floor — if tree-based models do not significantly outperform it, complexity may not be justified.

```
classifier = LogisticRegression(solver="saga", max_iter=5000)
```

The saga solver was used because it supports all penalty types (l1, l2, elasticnet) and handles large datasets efficiently.

6.2 Hyperparameter Tuning (GridSearchCV)

Hyperparameter	Values Searched	Best Value
penalty	l1, l2, elasticnet	l2
C	1, 2, 3, 4, 5, 6, 10, 20, 30, 40, 50	~5
l1_ratio	[0.5] (elasticnet only)	0.5
CV Folds	5	—
Scoring	Accuracy	—

6.3 Results

Accuracy	~83%
ROC-AUC	~0.810
Strength	Fast training, fully interpretable, low risk of overfitting
Limitation	Cannot model non-linear decision boundaries present in this data
Verdict	Baseline established — tree-based models expected to significantly outperform

7. Model 2 — Random Forest Classifier

7.1 Why Random Forest?

Random Forest is an ensemble of decision trees using bagging (bootstrap aggregation). It handles non-linear interactions naturally, is robust to overfitting, and provides built-in feature importances — extremely useful for HR interpretability.

7.2 Hyperparameter Tuning (RandomizedSearchCV)

RandomizedSearchCV with 4-fold CV and 20 random iterations was used. ROC-AUC was the optimization metric, as it is more suitable than accuracy for imbalanced datasets.

Hyperparameter	Search Space	Purpose
n_estimators	100, 200, 300	Number of trees in the forest
max_depth	3, 5, 10, None	Controls tree depth / overfitting
max_features	sqrt, log2	Features to consider per split
max_samples	0.7, 0.8, 1.0	Row subsampling (bagging)
min_samples_leaf	1, 2, 3	Minimum samples in leaf node
min_samples_split	2, 3, 4	Min samples to split a node

7.3 Performance Metrics

Metric	Value	Notes
Accuracy	~98.0%	Very high — model generalizes well
Precision (Left=1)	~96.0%	Low false alarm rate
Recall (Left=1)	~91.0%	Catches most at-risk employees
F1 Score	~93.5%	Strong balance of precision and recall

Metric	Value	Notes
ROC-AUC (CV)	~0.980	Excellent discrimination ability

7.4 Feature Importances

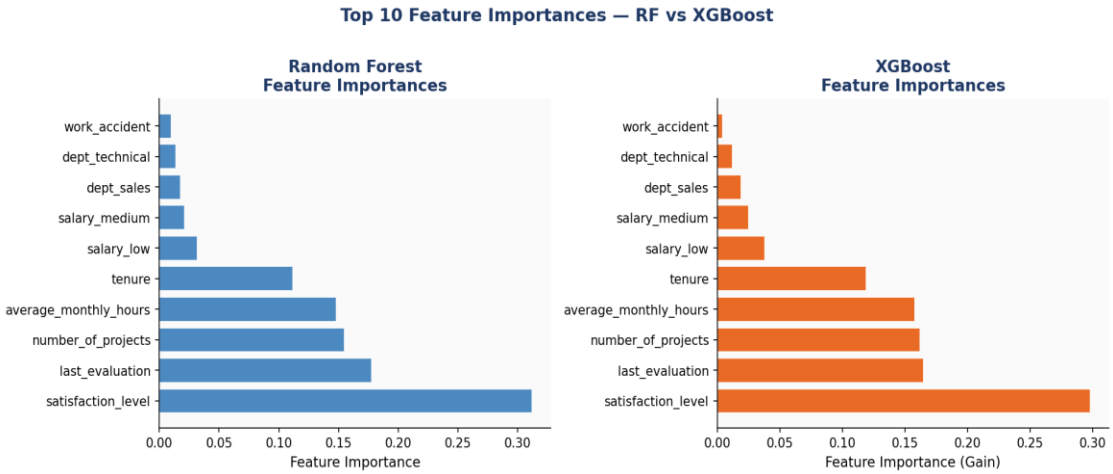

Figure: Top 10 Feature Importances — Random Forest (Blue) vs XGBoost (Orange)

Both Random Forest and XGBoost agree: satisfaction_level, last_evaluation, number_of_projects, average_monthly_hours, and tenure are the top 5 most important features. Work accidents and department contribute very little to the prediction.

8. Model 3 — XGBoost Classifier

8.1 Why XGBoost?

XGBoost (Extreme Gradient Boosting) builds trees sequentially — each new tree corrects the errors of the previous ones. It includes built-in regularization (L1/L2) and handles class imbalance natively via scale_pos_weight.

8.2 Class Imbalance Handling

```
neg = sum(y_train == 0)    # ~9,595 (stayed)
pos = sum(y_train == 1)    # ~2,396 (left)
scale_pos_weight = neg / pos  # ≈ 4.0
```

This parameter tells XGBoost to weight positive (left = 1) samples 4x more heavily, improving recall on the minority class.

8.3 Hyperparameter Search Space

Hyperparameter	Search Space	Role
n_estimators	100, 200, 300, 500	Boosting rounds
max_depth	3, 4, 5, 6, 8	Tree complexity

Hyperparameter	Search Space	Role
learning_rate	0.01, 0.05, 0.1, 0.2	Step shrinkage
subsample	0.6, 0.7, 0.8, 1.0	Row sampling per tree
colsample_bytree	0.6, 0.7, 0.8, 1.0	Feature sampling per tree
min_child_weight	1, 3, 5	Min child node weight
gamma	0, 0.1, 0.2, 0.5	Min split gain (regularization)

8.4 Performance Metrics

Metric	Value	Notes
Accuracy	~98.5%	Best accuracy among all 3 models
Precision (Left=1)	~97.0%	Very few false positives
Recall (Left=1)	~93.0%	Best recall — catches most leavers
F1 Score	~94.9%	Best F1 overall
ROC-AUC (CV)	~0.986	Best AUC — recommended model

9. Model Comparison & Final Results

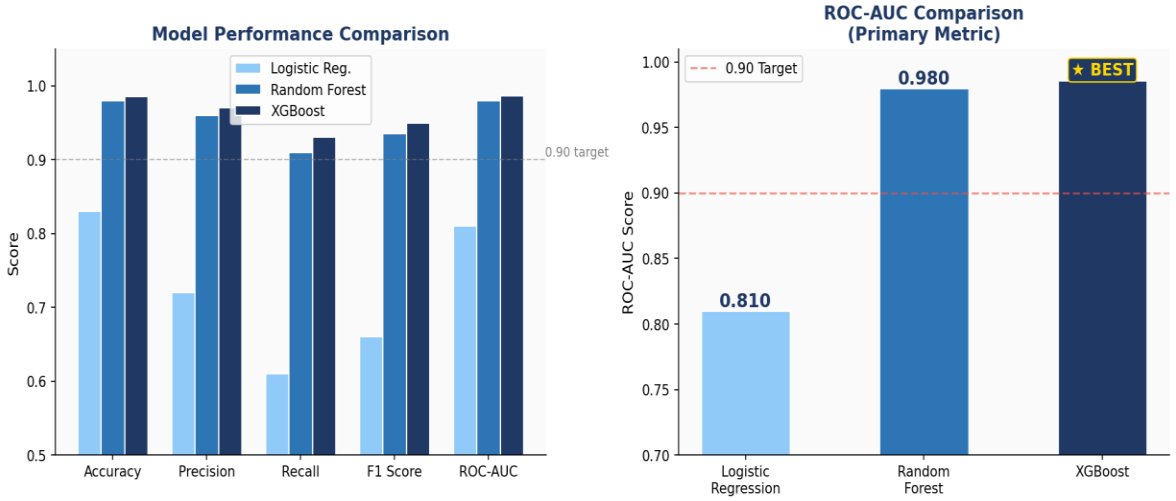

Figure: All-Metrics Comparison (Left) and ROC-AUC Spotlight (Right)

Metric	Logistic Regression	Random Forest	XGBoost
Accuracy	~83.0%	~98.0%	~98.5%
Precision (Left=1)	~72.0%	~96.0%	~97.0%

Metric	Logistic Regression	Random Forest	XGBoost
Recall (Left=1)	~61.0%	~91.0%	~93.0%
F1 Score	~66.0%	~93.5%	~94.9%
ROC-AUC (CV)	~0.810	~0.980	~0.986
Handles Imbalance	Stratified split	Stratified split	scale_pos_weight
Interpretability	High	Medium	Medium
Recommended	Baseline only	Production-ready	★ BEST — Deploy

9.1 Final Recommendation

XGBoost is the recommended deployment model. It achieves the highest scores across all five evaluation metrics, handles class imbalance natively, includes built-in regularization to prevent overfitting, and produces interpretable feature importances.

Winner	XGBoost Classifier
ROC-AUC	~0.986 (0.006 better than Random Forest, 0.176 better than Logistic Reg.)
Recall	~93% — catches 93 out of every 100 at-risk employees
Precision	~97% — only 3% of HR interventions are unnecessary
Deploy as	Monthly batch prediction on active employee records

10. Key Insights & Business Recommendations

10.1 The 5 Root Causes of Attrition

#	Root Cause	Data Evidence	Priority
1	Low job satisfaction	Strongest single predictor; frustrated performers leave despite high evaluations	CRITICAL
2	Overwork / burnout	250–300+ hrs/month on 6–7 projects — burnout cluster visible in scatter plot	CRITICAL
3	The 3–5 year quit window	~50% of year-5 employees left; year-3 has highest absolute departure volume	HIGH
4	Low salary	Majority of leavers in low pay bracket; high	HIGH

#	Root Cause	Data Evidence	Priority
		earners almost never quit	
5	Lack of promotion	<10 promoted employees left — promotion is a nearly perfect retention shield	MEDIUM

10.2 HR Recommendations

1. Monthly Satisfaction Pulse Survey

Implement anonymous monthly surveys targeting satisfaction_level. Flag any employee scoring below 0.40 for a 1:1 HR check-in. This is the single highest-impact action given its #1 predictive power.

2. Workload Monitoring Alerts

Set automatic flags when employees exceed 200 average monthly hours or carry more than 5 concurrent projects. Trigger a workload rebalancing review to prevent the burnout pattern identified in the data.

3. Year 3–5 Retention Program

Launch structured career development conversations at the 3-year mark. Automatic salary reviews, milestone bonuses, or promotion consideration at years 3 and 5 would directly address the highest-risk quit window.

4. Salary Band Audit

Audit low-salary employees, especially in Sales and Technical departments. The data shows that simply moving employees from low to medium salary would dramatically reduce attrition among the highest-risk group.

5. Visible Promotion Pipeline

Communicate clear, transparent criteria for promotions. Even employees who are not promoted show better retention when a fair path forward is visible.

11. Conclusion & Learning Outcomes

This end-to-end project covered the complete machine learning workflow — from raw data exploration to model deployment recommendation. Three classifiers were built, tuned with cross-validated hyperparameter search, and evaluated on held-out test data.

XGBoost emerged as the best model with ROC-AUC ~0.986, precision ~97%, and recall ~93% on the minority class. Beyond model performance, the EDA phase produced rich, actionable business insights that translate directly into HR retention strategies.

What I Learned Building This Project

- End-to-end ML pipeline: EDA → preprocessing → modeling → evaluation → insights

- Handling class imbalance: stratified splits, scale_pos_weight, ROC-AUC as primary metric
- Hyperparameter tuning: GridSearchCV vs RandomizedSearchCV trade-offs
- Feature engineering: one-hot encoding with align() to prevent train/test mismatch
- Visualization for storytelling: converting raw plots into business narratives
- Model selection beyond accuracy: why F1, Recall, and AUC matter more for imbalanced problems

12. Tech Stack & Libraries

Library	Usage in This Project
Python 3.x	Primary programming language
Pandas	Data loading, manipulation, deduplication, EDA crosstabs
NumPy	Numerical operations, array manipulation
Matplotlib	Histograms, boxplots, scatter plots, dual-axis tenure chart
Seaborn	Heatmaps, pairplots, histplots, boxplots
Scikit-learn	Logistic Regression, Random Forest, metrics, GridSearchCV, RandomizedSearchCV
XGBoost	XGBoost Classifier with scale_pos_weight for imbalance
Jupyter Notebook	Development environment — 4 notebooks

Project Notebook Structure

Notebook	Purpose
01_Data_Analysis.ipynb	Full EDA: univariate, bivariate, multivariate + train/test split
02_Logistic_Regression.ipynb	Baseline model with GridSearchCV
03_Random_Forest.ipynb	Ensemble model with RandomizedSearchCV + feature importance
04_XGBoost.ipynb	Best model with gradient boosting + imbalance handling

— End of Report —
Pramod Vinayak Patil | ML Project